

Wrangling

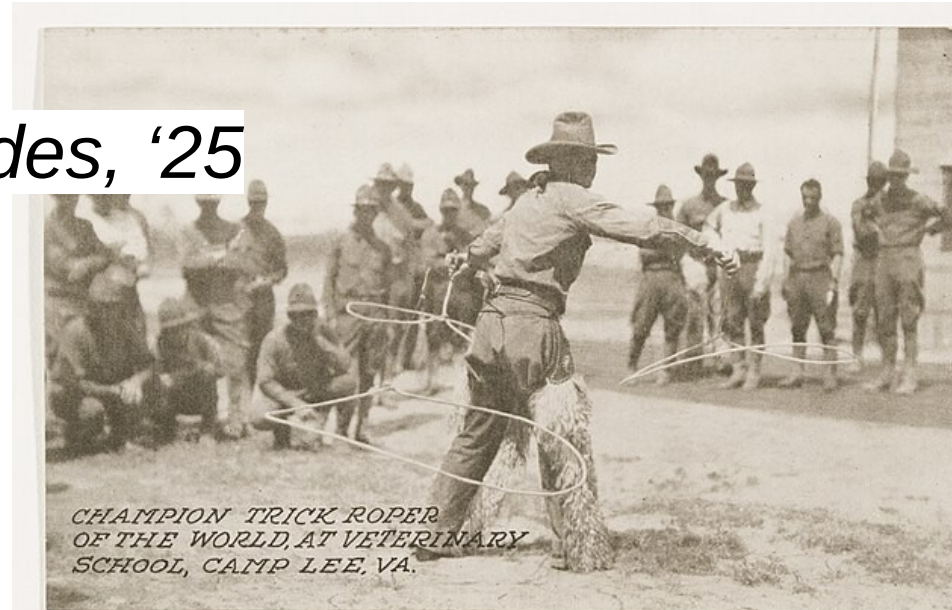
CIS 5450, Spring 2026

Harry Smith

Adapted from Ryan Marcus' slides, '25



**Penn
Engineering**
UNIVERSITY of PENNSYLVANIA



Outline

- Course logistics
- Brief review
- Types of data (“data modalities”)
- What is wrangling?
- Pandas (not the cute kind)

Outline

- **Course logistics**
- Brief review
- Types of data (“data modalities”)
- What is wrangling?
- Pandas (not the cute kind)

Upcoming Deadlines

- HW0 due Jan 29th at 10:00pm
 - Recitation on Friday, Jan 23rd will be Open OH for submission/PennGrader help.

Head TAs

- Vivian Ziao
- Aeshon Balasubramanian
- Xiang Chen

Recitations

- First “office hours” recitation this Friday
- Lecture attendance is encouraged
- “Homework” recitation attendance is *strongly* encouraged
 - First “Homework” recitation is on Jan 30th for HW1 (next assignment)

Outline

- Course logistics
- **Brief review**
- Types of data (“data modalities”)
- What is wrangling?
- Pandas (not the cute kind)



Raw Data

Clean Data

**Extracted
Features**

Models

Data collected from
the world

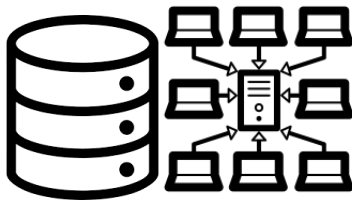
Data in a meaningful
or useful form

Measurable
characteristics

Descriptive or
predictive models

Part 1

Now!



How does it all work?
Design of data systems

Part 2

Part 3

The Five Vs (in readable form)

- Each “V” represents a challenge of Big Data we have to solve.



Volume

There's a lot of data, and we need a strategy for managing it. A single Excel sheet won't work.



Velocity

Speed matters. If our analysis takes 1 week, then our insights are always 1 week behind.



Value

Data without insights are a liability, not an asset. How can we get *valuable* info out of our data?



Variety

Data is heterogenous; some structured (rows/cols), some unstructured (raw text). We have to handle both.



Veracity

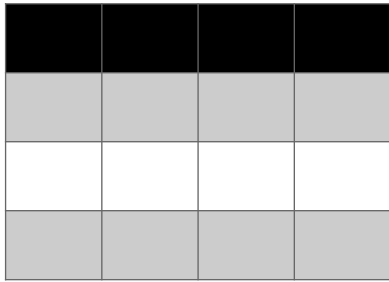
Data is *heteroskedastic*: data quality can be poor, data can be missing. How will you clean the data?

Outline

- Course logistics
- Brief review
- **Types of data (“data modalities”)**
- What is wrangling?
- Pandas (not the cute kind)

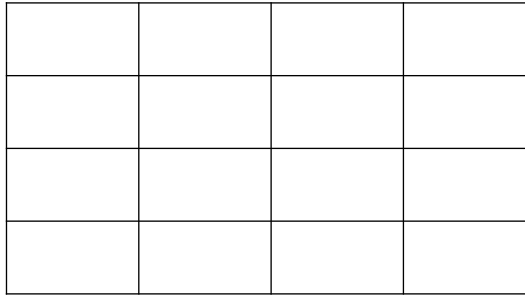
Data Modalities

- “Modality” = form, format, type, etc.



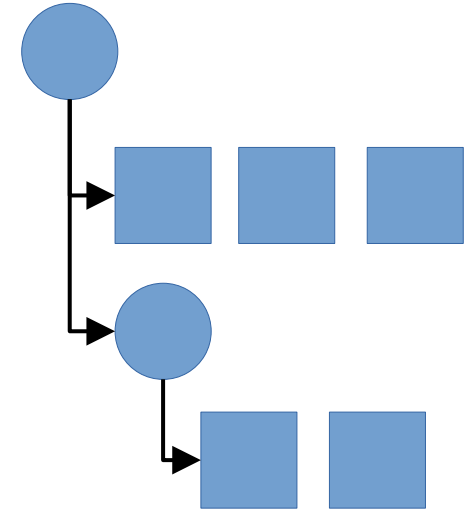
Tables

Relations
Spreadsheets
Dataframes



Arrays

Tensors
Matrix



Document

Hierarchy
Tree
HTML / XML

Can you think of examples of data sources that might take each of these forms?

Tables

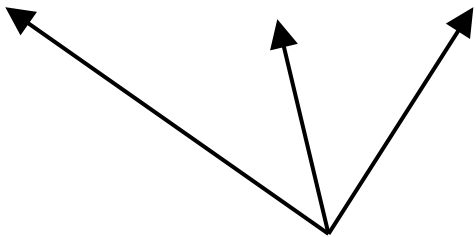
- Rows (tuples / facts)
- Columns (attributes)
- Main form of data in SQL databases

First	Last	YoE
Ryan	Marcus	10
Harry	Smith	13

Tables

- Rows (tuples / facts)
- Columns (attributes)
- Main form of data in SQL databases

First	Last	YoE
Ryan	Marcus	10
Harry	Smith	13

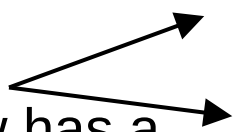


Each column has a uniform type

Tables

- Rows (tuples / facts)
- Columns (attributes)
- Main form of data in SQL databases

Each row has a value for each column



First	Last	YoE
Ryan	Marcus	10
Harry	Smith	13

Tables

- Rows (tuples / facts)
- Columns (attributes)
- Main form of data in SQL databases

First	Last	YoE
Ryan	Marcus	10
Harry	Smith	13

Operations

Select specific rows by property
Take the average of a column
Group the table by some property
Join the table with another table

Arrays

- Uniform datatype
- Rows and columns are data-dependent

2.5	4.2	5.1	1.9
4.5	3	2.3	1.2
1.2	2.4	4.5	3.2
56.5	4.2	7.9	5.6

Arrays
Tensors
Matrix

Arrays & Matrices are often *mathematical* objects. What are some operations available on data in this form?

Arrays

- Uniform datatype
- Rows and columns are data-dependent

2.5	4.2	5.1	1.9
4.5	3	2.3	1.2
1.2	2.4	4.5	3.2
56.5	4.2	7.9	5.6

Operations

Matrix multiply
Element-wise multiply
Dot product
Solve
Compute rank

Arrays
Tensors
Matrix

Documents

- Nested sequence of *maps* and *lists*

```
{  
  "key1": ["value1", "value2"],  
  "key2": { "subkey1": "value"}  
}
```

JSON

```
<person auth="true">  
  <first>Harry</first>  
  <last>Smith</last>  
  <contact>  
    <email>sharry@seas.upenn.edu</email>  
  </contact>  
</person>
```

XML

JSON as API Response

▼ REQUEST SAMPLE

cURL

Wget

HTTPIe

```
1  curl --request GET \  
2    --url  
https://api.spotify.com/v1/arti  
sts/0Tn0YISbd1XYRBk9myaseg \  
3    --header 'Authorization:  
Bearer  
1P0dFZRZbvb...qqillRxMr2z'
```



• RESPONSE SAMPLE

```
1  {  
2    "external_urls": {  
3      "spotify": "string"  
4    },  
5    "followers": {  
6      "href": "string",  
7      "total": 0  
8    },  
9    "genres": ["Prog rock",  
"Grunge"],  
10   "href": "string",  
11   "id": "string",  
12   "images": [  
13     {  
14       "url":  
"https://i.scdn.co/image/ab67616d0  
0001e02ff9ca10b55ce82ae553c8228",  
15       "height": 300,  
16       "width": 300  
17     }  
18   ],  
19   "name": "string",  
20   "popularity": 0,  
21   "type": "artist",  
22   "uri": "string"  
23 }
```

Keys, like
tabular
column
labels,
define what
attributes
the data
contains.

● RESPONSE SAMPLE

```
1  {  
2    "external_urls": {  
3      "spotify": "string"  
4    },  
5    "followers": {  
6      "href": "string",  
7      "total": 0  
8    },  
9    "genres": ["Prog rock",  
10   "Grunge"],  
11    "href": "string",  
12    "id": "string",  
13    "images": [  
14      {  
15        "url":  
16        "https://i.scdn.co/image/ab67616d0  
17        0001e02ff9ca10b55ce82ae553c8228",  
18        "height": 300,  
19        "width": 300  
20      }  
21    ],  
22    "name": "string",  
23    "popularity": 0,  
24    "type": "artist",  
25    "uri": "string"  
26  }
```

Values
define the
contents of
the
attributes for
a particular
object.

RSS: it's XML!

```
▼<rss xmlns:atom="http://www.w3.org/2005/Atom"
  xmlns:content="http://purl.org/rss/1.0/modules/content/"
  xmlns:googleplay="http://www.google.com/schemas/play-podcasts/1.0"
  xmlns:itunes="http://www.itunes.com/dtds/podcast-1.0.dtd"
  xmlns:media="http://search.yahoo.com/mrss/"
  xmlns:podcast="https://podcastindex.org/namespace/1.0" version="2.0"
  ▼<channel>
    <atom:link href="https://feeds.simplecast.com/S15CSM3S" rel="self"
      type="application/atom+xml"/>
    <atom:link xmlns="http://www.w3.org/2005/Atom"
      href="https://simplecast.superfeedr.com" rel="hub"/>
    <generator>https://simplecast.com</generator>
    <title>The Daily</title>
    <description>This is what the news should sound like. The biggest
      time, told by the best journalists in the world. Hosted by Michael
      Abrams and Natalie Kitroeff. Twenty minutes a day, six days a week.
      Subscribe today at nytimes.com/podcasts or on Apple Podcasts and
      also subscribe via your favorite podcast app here https://www.nytimes.com/
      access/audio?source=podcatcher.</description>
    <copyright>© 2020–2021 THE NEW YORK TIMES COMPANY; The New York Times
      use of RSS feeds for personal use in a news reader or as part of a
      blog, subject to your agreement to our Terms of Service.</copyright>
    <language>en</language>
    <pubDate>Sun, 18 Jan 2026 11:00:00 +0000</pubDate>
```

Data Modalities

- Any data can be represented in any form
 - We'll show this rigorously later
- Different tools work with different forms
- Choosing the right modality is context-dependent
 - Pros and cons of different formats later

Outline

- Course logistics
- Brief review
- Types of data (“data modalities”)
- **What is wrangling?**
- Pandas (not the cute kind)

Wrangling

- Getting data from a “raw” format into a “processed” format
 - Obviously, “raw” and “processed” are context-dependent
- Often the *most time consuming part!*
- Example: collecting all course grades into a spreadsheet
- Example: managing mega-mart inventory

In the industry, information from vendors is notoriously unreliable, but merchandising assistants were often not experienced enough to challenge vendors on the accuracy of the product information they provided. (The staff were also working against the countdown to opening.)

“There was never any talk about accuracy,” says a former employee. “You had these people we hired, straight out of school, pressured to do this insane amount of data entry, and nobody told them it had to be right.”

Worse, the company hadn’t built a safety net into SAP at this point; the system couldn’t notify users about data entry errors. The investigative team estimated information in the system was accurate about 30% of the time. In the U.S., it’s between 98% and 99%.

Wrangling

- Our working definition:
Transforming data to be ready for analysis
- ETL:
 - **E**xtract the data
 - **T**ransform (and clean) the data
 - **L**oad the data into whatever is doing your analysis

Outline

- Course logistics
- Brief review
- Types of data (“data modalities”)
- What is wrangling?
- **Pandas (not the cute kind)**

Pandas

- Python package commonly used for data tasks
- Available (almost) everywhere!
- **Panel Data:** tables!

- Goal: figure out how to fund a venture fund
- Data in CSV

```

1 permalink,name,homepage_url,category_list,market,funding_total
2 /organization/waywire,#waywire,http://www.waywire.com,|
  Entertainment|Politics|Social Media|News|, News ,
  1 750 000 ,acquired,USA,NY,New York City,New
  York,1,01/06/2012,2012-06,2012-
  Q2,2012,30/06/2012,30/06/2012
3 /organization/tv-communications,&TV Communications,http://
  enjoyandtv.com,|Games|,Games,
  4 000 000 ,operating,USA,CA,Los Angeles,Los
  Angeles,2,,,,,04/06/2010,23/09/2010
4 /organization/rock-your-paper,'Rock' Your Paper,http://
  www.rockyourpaper.org,|Publishing|Education|,Publishing,
  40 000 ,operating,EST,,Tallinn,Tallinn,1,26/10/2012,2012-10-
  Q4,2012,09/08/2012,09/08/2012
5 /organization/in-touch-network,(In)Touch Network,http://
  www.InTouchNetwork.com,|Electronics|Guides|Coffee|
  Restaurants|Music|iPhone|Apps|Mobile|iOS|E-
  Commerce|,Electronics,
  1 500 000 ,operating,GBR,,London,London,1,01/04/2011,2011-0
  Q2,2011,01/04/2011,01/04/2011
6 /organization/n-plusn,+n (PlusN),http://plusn.com,|
  Software|, Software , 1 200 000 ,operating,USA,NY,New York
  City,New York,2,01/01/2012,2012-01,2012-
  Q1,2012,29/08/2012,04/09/2014

```



```
import pandas as pd
```

Python library = “pandas”
Alias we will use = “pd”
(this is standard)

```
✓ 0s [1] import pandas as pd
```

```
✓ 3s [2] df = pd.read_csv("https://rmarcus.info/Companies.csv")  
df
```



	permalink	name	homepage_url	category_list
0	/organization/waywire	#waywire	http://www.waywire.com	Entertainment Politics Social Media News
1	/organization/tv-communications	&TV Communications	http://enjoyandtv.com	Games
2	/organization/rock-your-paper	'Rock' Your Paper	http://www.rockyourpaper.org	Publishing Education
3	/organization/in-touch-network	(In)Touch Network	www.InTouchN	
4	/organization/n-plusn	+n (PlusN)	http://plusn.com	Software



DataFrames are tables; load one with pd.read_csv()

df.columns



```
Index(['permalink', 'name', 'homepage_url', 'category_list',  
      'market',  
      'funding_total_usd', 'status', 'country_code',  
      'state_code', 'region',  
      'city', 'funding_rounds', 'founded_at', 'founded_month',  
      'founded_quarter', 'founded_year', 'first_funding_at',  
      'last_funding_at', 'clean_funding_total_usd'],  
      dtype='object')
```

Property of dataframe
“index” (list) of columns

✓ 0s [23] df.columns

⇒ Index(['permalink', 'name', 'homepage_url', 'category',
'funding_total_usd', 'status', 'country_code',
'city', 'funding_rounds', 'founded_at', 'founder',
'founded_quarter', 'founded_year', 'first_funding_at',
'last_funding_at'],
dtype='object')

Inspect a subset of the
dataframe (list of columns)

✓ 0s [24] df[["name", "market", "funding_total_usd"]]



name market funding_total_usd



Hmm... what's up with these
spaces?

News 1 750 000

Games 4 000 000

Publishing 40 000

3 (In)Touch Network Electronics 1 500 000

4 +n (PlusN) Software 1 200 000

...

47753 Zzish Education 320 000

47754 77Node Science and Technology Enterprise Software 1587 301

Sort by funding, largest values
first, look at the top 10

```
(df[["name", "market", "funding_total_usd"]]  
  .sort_values("funding_total_usd", ascending=False)  
  .head(10))
```

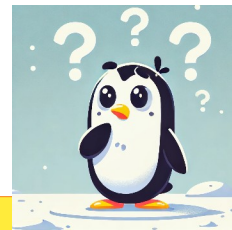
The parentheses are so we
can call the .sort_values()
and .head() methods on
different lines for legibility.

```
(df[["name", "market", "funding_total_usd"]]  
  .sort_values("funding_total_usd", ascending=False)  
  .head(10))
```



	name	market	funding_total_usd
4370	Bayhill Therapeutics	Biotechnology	9 999 999
44445	Victory Pharma	Biotechnology	9 999 997
29517	OpenDoor	E-Commerce	9 990 000
32049	PowerCell Sweden	Clean Technology	9 981 363
42092	Topcom Europe	Hardware + Software	9 980 000
20302	Intellectap	Consulting	9 971
13796	Everyone Counts	Software	9 957 650
35894	ScriptRock	Intelligent Systems	9 952 199
10434	CyPhy Works	Manufacturing	9 950 002

Sort by funding, largest values first, look at the top 10



What happened here? 🤔

0s

```
(df[["name", "market", "funding_total_usd"]
.sort_values("funding_total_usd", ascending=False)
.head(10))
```

	name	market	funding_total_usd
4370	Bayhill Therapeutics	Biotechnology	9 999 999
44445	Victory Pharma	Biotechnology	9 999 997
29517	OpenDoor	E-Commerce	9 990 000
32049	PowerCell Sweden	Clean Technology	9 981 363
42092	Topcom Europe	Hardware + Software	9 980 000
20302	Intellect	Consulting	9 971
13796	Everyone Counts	Software	9 957 650
35894	ScriptRock	Intelligent Systems	9 952 199
10434	CyPhy Works	Manufacturing	9 950 002
11955	drop.io	Email	9 950 000

what datatype does pandas think we have?

0s

```
[36] df[["name", "market", "funding_total_usd"]].dtypes
```

	0
name	object
market	object
funding_total_usd	object

“object” means “could be anything” / text

✓
0s [38] df["funding_total_usd"].head()



funding_total_usd

0 1 750 000

1 4 000 000

2 40 000

3 1 500 000

It looks like the format is spaces where there are usually commas, we want neither

✓
0s [38] df["funding_total_usd"].head()



	funding_total_usd
0	1 750 000
1	4 000 000
2	40 000
3	1 500 000
4	1 200 000

dtype: object

It looks like the format is spaces where there are usually commas, we want neither

Write a function to clean each row, add docs and tests

✓
0s import re

```
def funding_str_to_int(s):  
    # s is something like "170 000" and we want 170000  
    s = re.sub("[^0-9]", "", s)  
    if len(s) == 0:  
        return None  
    else:  
        return int(s)
```

```
assert funding_str_to_int("170 000") == 170000  
assert funding_str_to_int("0") == 0  
assert funding_str_to_int("") == None
```

```
df["clean_funding_total_usd"] = df["funding_total_usd"].str.replace("[^0-9]", "", regex=True)
df["clean_funding_total_usd"] = df["clean_funding_total_usd"].replace("", "0")
df["clean_funding_total_usd"] = df["clean_funding_total_usd"].astype(int)
```

1. Remove all spaces using regular expressions.

2. Replace all empty strings with default "0"

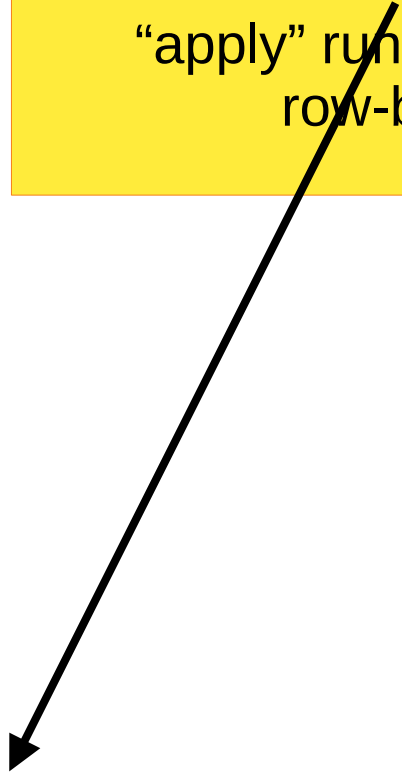
3. Force pandas to treat the Series as integers

Make a new column by
applying our function to an
existing function

```
# alternatively...  
import re  
def funding_str_to_int(s):  
    s = re.sub("[^0-9]", "", s)  
    if len(s) == 0:  
        return None  
    else:  
        return int(s)
```

```
df["clean_funding_total_usd"] = df["funding_total_usd"].apply(funding_str_to_int)
```

“apply” runs a function
row-by-row



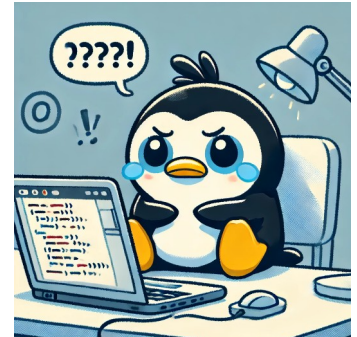
Look at our top 10 again...

```
[42] df["clean_funding_total_usd"] = df["funding_total_usd"].
```

```
(df[["name", "market", "clean_funding_total_usd"]]  
sort_values("clean_funding_total_usd", ascending=False))
```

Missing values!

	name	market	clean_funding_total_usd
6984	Cardinal Health	NaN	7.879506e+10
44270	Verizon Communications	Mobile	3.007950e+10
35658	Sberbank	Finance	5.800000e+09
8370	Clearwire	Internet	5.700000e+09
7703	Charter Communications	NaN	5.162513e+09
14807	First Data Corporation	Trading	3.500000e+09
8842	COFCO	NaN	3.200000e+09
36990	sigmacare	Health and Wellness	2.600000e+09
14216	Facebook	Communities	2.425700e+09
7078	Carestream	Biotechnology	2.400000e+09



```
✓ [11] df.loc[df["name"] == "Cardinal Health", "market"] = "Health"
0s
```

```
✓ [13] df["market"].isnull().value_counts()
0s
```



	count
market	
False	42466
True	5292

dtype: int64

Manually correct this missing value

```
✓ [14] df["clean_funding_total_usd"].isnull().value_counts()
0s
```



	count
clean_funding_total_usd	
False	39726
True	8032

dtype: int64

Check to see how prevalent missing values are in the data

✓
0s

```
▶ top_seg = (df[["market", "clean_funding_total_usd"]]  
             .groupby("market")  
             .sum()  
             .sort_values("clean_funding_total_usd", ascending=False)  
             .head(25))  
top_seg.head(5)
```



clean_funding_total_usd



market



Biotechnology

7.161303e+10

Mobile

4.936859e+10

Software

4.200789e+10

Clean Technology

3.747349e+10

Health Care

2.837460e+10

✓
0s

```
▶ top_seg = (df[["market", "clean_funding_total_usd"]]  
             .groupby("market")  
             .sum()  
             .sort_values("clean_funding_total_usd", ascending=False)  
             .head(25))  
top_seg.head(5)
```



clean_funding_total_usd	
market	
Biotechnology	7.161303e+10
Mobile	4.936859e+10
Software	4.200789e+10
Clean Technology	3.747349e+10
Health Care	2.837460e+10



“Group” multiple rows into a single row, based on a “key”

✓
0s

```
▶ top_seg = (df[["market", "clean_funding_total_usd"]]  
             .groupby("market")  
             .sum()  
             .sort_values("clean_funding_total_usd", ascending=False)  
             .head(25))  
top_seg.head(5)
```



clean_funding_total_usd



market	
Biotechnology	7.161303e+10
Mobile	4.936859e+10
Software	4.200789e+10
Clean Technology	3.747349e+10
Health Care	2.837460e+10

For the non-key columns,
aggregate via sum operator

✓
0s

```
▶ top_seg = (df[["market", "clean_funding_total_usd"]]  
             .groupby("market")  
             .sum()  
             .sort_values("clean_funding_total_usd", ascending=False)  
             .head(25))  
top_seg.head(5)
```



clean_funding_total_usd



market



Biotechnology

7.161303e+10

Mobile

4.936859e+10

Software

4.200789e+10

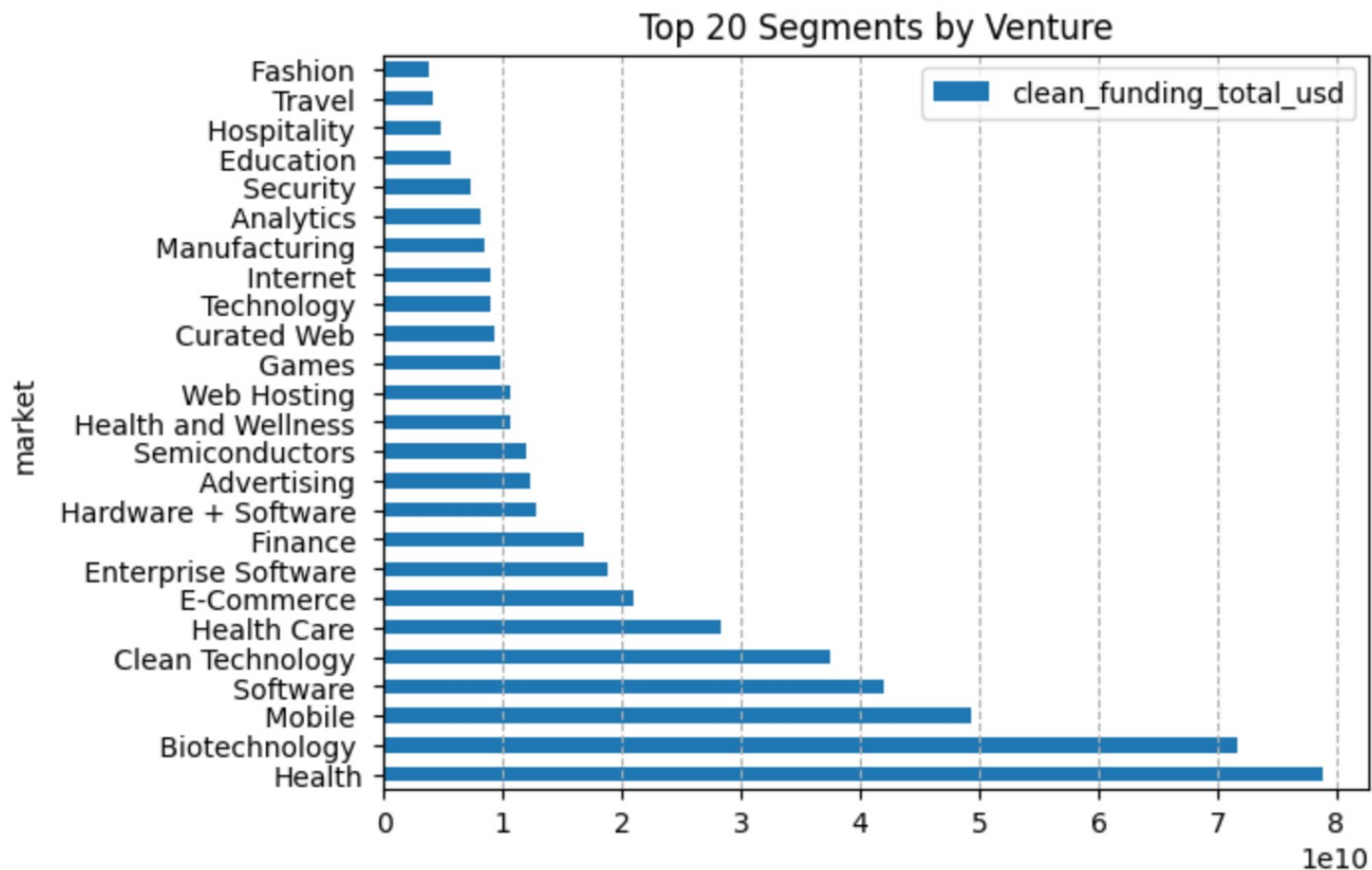
Clean Technology

3.747349e+10

Health Care

2.837460e+10

```
✓ [17] ax = top_seg.plot.barh()  
1s ax.set_title("Top 20 Segments by Venture")  
ax.grid(axis="x", ls="--")
```



Pandas and Polars

- Pandas is the “Swiss army knife” of data processing
- Polars, a performance-focused pandas alternative
- Question: which airline cancels the most flights?



```
✓ 9s [2] !pip install -U altair polars
import pandas as pd
import polars as pl
```

Polars is not installed by default, so we install it with pip

```
✓ 21s [3] from google.colab import drive
drive.mount('/content/drive')
```

Make our Google Drive visible to Colab

Mounted at /content/drive

```
✓ 0s # Get the flights.csv file from here: https://drive.google.com/...
# ... and add it to your Google Drive. Update this variable to the new path
# (e.g., remove "colab_data" if you didn't create a folder with that name)
PATH = "/content/drive/MyDrive/colab_data/flights.csv"
```

```
✓ 23s [6] df = pd.read_csv(PATH)
df
```

Read the data

<ipython-input-6-7b7b758c51cc>:1: DtypeWarning: Columns (...)
df = pd.read_csv(PATH)

	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_	
0	2015	1	1	4	AS	98	N407AS
1	2015	1	1	4	AA	2336	N3KUAA
2	2015	1	1	4	US	840	N171US

```
[7] total_flights = df["AIRLINE"].value_counts().to_frame()  
total_flights
```

Compute total number
of flights for each airline



count



AIRLINE

WN 1261855

DL 875881

AA 725984

OO 588353

EV 571977

UA 515723

MQ 294632

B6 267048

US 198715

AS 172521

```
[8] cancelled_flights = df[df["CANCELLED"] == 1]["AIRLINE"].value_counts().to_fr  
cancelled_flights
```



count



AIRLINE

WN 16043

EV 15231

MQ 15025

AA 10919

OO 9960

UA 6573

B6 4276

US 4067

DL 3824



Compute total number of flights
for each airline

```

df = pd.read_csv(PATH)
total_flights = df["AIRLINE"].value_counts().to_frame()
total_flights = total_flights.rename(columns={"count": "total"})

cancelled_flights = df[df["CANCELLED"] == 1]["AIRLINE"].value_counts().to_frame()
cancelled_flights = cancelled_flights.rename(columns={"count": "cancelled"})

df = total_flights.merge(cancelled_flights, on="AIRLINE")
df["perc cancelled"] = (df["cancelled"] / df["total"])*100
df = df.sort_values("perc cancelled", ascending=False)
df

```

“Join” tables on a common column

Compute the percentage

<ipython-input-9-3002dae07164>:1: DtypeWarning: Columns (7,8) have mixed types. Si
df = pd.read_csv(PATH)

total cancelled perc cancelled

AIRLINE

MQ 294632 15025 5.099582

EV 571977 15231 2.662869

US 198715 4067 2.046650

NK 117379 2004 1.707290

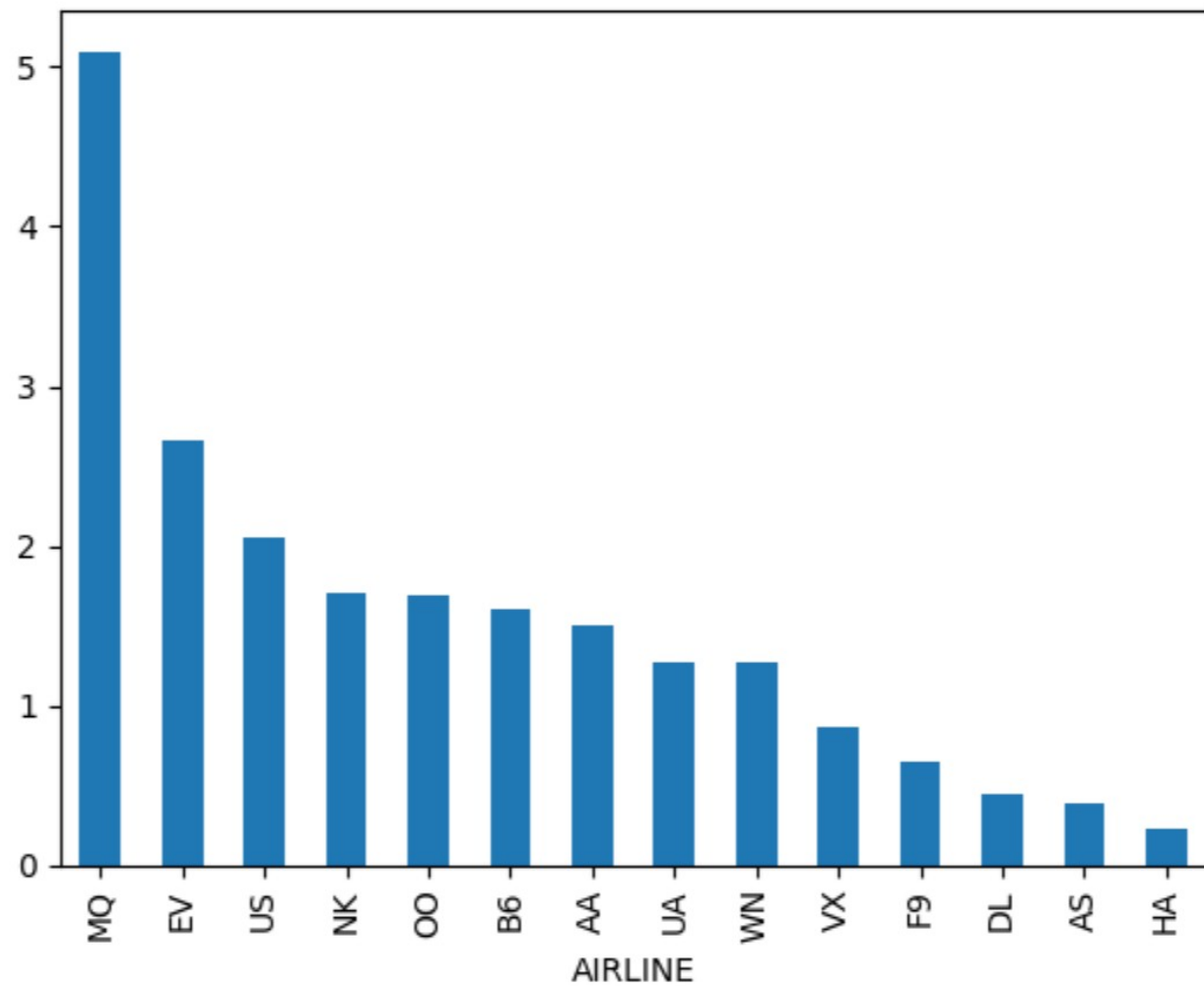
OO 588353 9960 1.692861

Sort

```
[11] df["perc cancelled"].plot.bar()
```



<Axes: xlabel='AIRLINE'>



[12] %%time

special timing command

```
df = pd.read_csv(PATH)
total_flights = df["AIRLINE"].value_counts().to_frame()
total_flights = total_flights.rename(columns={"count": "total"})

cancelled_flights = df[df["CANCELLED"] == 1]["AIRLINE"].value_counts().to_frame()
cancelled_flights = cancelled_flights.rename(columns={"count": "cancelled"})

df = total_flights.merge(cancelled_flights, on="AIRLINE")
df["perc cancelled"] = (df["cancelled"] / df["total"])*100
df = df.sort_values("perc cancelled", ascending=False)
df
```



```
<timed exec>:1: DtypeWarning: Columns (7,8) have mixed types. Specify dtype option
CPU times: user 15.1 s, sys: 4.98 s, total: 20.1 s
Wall time: 22 s
```

22 seconds



```
%%time
```

```
df = pl.read_csv(PATH)
```

```
total_flights = df["AIRLINE"].value_counts()
```

```
total_flights = total_flights.rename({"count": "total"})
```

```
cancelled_flights = df.filter(pl.col("CANCELLED") == 1)["AIRLINE"].value_counts()
```

```
cancelled_flights = cancelled_flights.rename({"count": "cancelled"})
```

```
df = total_flights.join(cancelled_flights, on="AIRLINE")
```

```
df = df.with_columns((pl.col("cancelled")  
                    / pl.col("total")*100).alias("perc cancelled"))
```

```
df = df.sort("perc cancelled", descending=True)
```

```
df
```

use polars (pl)
instead of pandas (pd)

some small syntax
differences



```
CPU times: user 10.8 s, sys: 776 ms, total: 11.6 s
```



```
%%time
```

```
df = pl.read_csv(PATH)
```

```
total_flights = df["AIRLINE"].value_counts()
```

```
total_flights = total_flights.rename({"count": "total"})
```

```
cancelled_flights = df.filter(pl.col("CANCELLED") == 1)["AIRLINE"].value_counts()
```

```
cancelled_flights = cancelled_flights.rename({"count": "cancelled"})
```

```
df = total_flights.join(cancelled_flights, on="AIRLINE")
```

```
df = df.with_columns((pl.col("cancelled")  
    / pl.col("total")*100).alias("perc cancelled"))
```

```
df = df.sort("perc cancelled", descending=True)
```

```
df
```

use polars (pl)
instead of pandas (pd)

some small syntax
differences



```
CPU times: user 10.8 s, sys: 776 ms, total: 11.6 s
```

```
Wall time: 7.38 s
```

7 seconds!